

CAS4133 LLM · 기말 대비 · Chapter 14 · Special Lecture

# Chapter 14. AI-Generated Text Detection

(Special Lecture)

LLM이 생성한 텍스트를 어떻게 탐지하는가 — Zero-shot 탐지(DetectGPT → Fast-DetectGPT), 학습 기반 탐지(ReMoDetect), 그리고 생성 단계에서 패턴을 심는 Watermarking(Hard Red List → Soft Method)

 강의일: Special Lecture #14 예상 비중: 중상 (기말 신규 범위) 분량: 본문 8섹션 + 문제 14

## 출제 방식 공지 (교수 안내)

Special Lecture 두 개(#13 Jailbreak + #14 AI Text Detection)는 **하나의 챕터(Problem)로 묶여 multiple choice 문제로만 출제**된다. True/False가 아닌 **"Pick all that are correct" 형식**이므로 보기 4개를 각각 독립적으로 O× 판정하는 연습이 핵심. 두 특강 개념이 한 문제 안에 섞여 나올 수 있으니(예: DetectGPT의 perturbation과 COLD-Attack의 Langevin 차이) **#13 정리**와 묶어서 공부할 것.

## 목차

1. Motivation — LLM 텍스트 오용 위험과 탐지 서비스 (GPTZero)
2. DetectGPT ① — Negative Curvature 가설과 Perturbation Discrepancy Score
3. DetectGPT ② — Curvature 해석 (Taylor Expansion + Hutchinson's Estimator → Hessian Trace)
4. DetectGPT ③ — 실험 (AUROC, Supervised 비교, Ablation)
5. Fast-DetectGPT — Conditional Probability Curvature와 340배 가속
6. ReMoDetect — Reward Model로 Aligned LLM 텍스트 탐지

7. Watermarking — Hard Red List, One Proportion z-Test, Soft Method

8. 탐지 패러다임 총비교 — Zero-shot vs Trained vs Watermarking

9. 예상문제 (T/F 8 + Pick-all 6)

LLM은 이제 **fluent and convincing text**를 생성하며, 그 품질은 **human specialists에 견줄 만한 (comparable to human specialists)** 수준이다. 요약, 이메일 응답 등 다양한 응용에서 이득을 주지만, 생성된 텍스트가 **human-written content와 구분하기 어렵다(difficult to distinguish)**는 점이 문제가 된다. 이 현상은 LLM 발전(GPT-5, Claude 4 등)과 함께 더 심해질 것이고, 동시에 **misuse(오용)의 잠재력**도 커진다.

## 구체 사례: LLM이 생성한 논문 리뷰

- 리뷰어가 시간을 아끼려고 LLM(ChatGPT)에 정해진 포맷으로 리뷰를 자동 생성시켜 그대로 제출하는 사례 — 저자가 수개월~수년을 쏟은 논문에 성의 없는 낮은 점수 리뷰가 달리는 나쁜 현상.
- 커뮤니티의 대응: **ICLR** 등 주요 ML 학회가 LLM 사용에 대한 **specific guidelines**를 도입. 하지만 가이드라인만으로는 불충분 — **리뷰어가 실제로 LLM을 썼는지 어떻게 알 수 있는가?** → **quantitative 탐지 방법(quantitative method to detect)**이 필요. 이것이 AI text detection의 핵심 동기.

### 📖 교수 강조 (강의 발언)

"This is a real example that I reviewed this year. ... Just imagine that you got this kind of very bad review with a low score while you are spending three or four months, or even years, on the paper." — 그리고 "Maybe it is a more interesting research question for me because I need to detect whether homework or assignments are generated by ChatGPT or not." (교수 본인의 과제 채점 동기 언급)

## 상용 탐지 서비스: GPTZero

- AI-text detector 서비스가 활발히 개발 중이며, 대표 예가 **GPTZero** (gptzero.me).
- 교수의 데모: LLM이 생성한 리뷰를 넣으면 전체가 AI 생성임에도 **일부만 높은 AI 확률로 부분 탐지** → "not perfect, but not bad". 교수가 직접 쓴 리뷰를 넣으면 거의 100% human으로 판정 → 사람/AI 구분 능력도 꽤 양호.

이 장의 큰 그림: 탐지 접근법 3갈래 — ① **Zero-shot (training-free)**: DetectGPT, Fast-DetectGPT — 모델 likelihood의 기하학적 성질 이용. ② **Training-based**: supervised classifier(RoBERTa 파인튜닝), ReMoDetect(reward model 파인튜닝). ③ **Watermarking**: 탐지가 아니라 **생성 시점에 패턴을 심어** 나중에 통계 검정으로 확인.

⚠ **함정 포인트 (T/F 대비)**

- "GPTZero perfectly identifies every part of an LLM-generated passage as AI-written." → **False**. In the professor's demo, GPTZero only partially detected the fully LLM-generated review — it is "not perfect, but not bad."
- "Since conferences like ICLR introduced guidelines on LLM usage, quantitative detection methods are unnecessary." → **False**. Guidelines alone cannot verify whether a reviewer actually used an LLM; that is exactly the motivation for quantitative detection.

## DetectGPT ① — Negative Curvature 가설과 Perturbation Discrepancy Score

Mitchell et al., "DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature", ICML 2023 — 이 분야의 pioneer work.

### 핵심 관찰 (Observation)

#### 📖 관찰

LLM-generated texts tend to occupy negative curvature regions of the model's log probability function. 생성 입력  $x$ 를 source model  $p_\theta$ 의 log likelihood 표면에 찍어보면, AI 생성 텍스트는 높은 확률로 local maxima 근처에 위치하지만, human-written text는 그렇다고 말하기 어렵다.

### Goal · Assumption · Hypothesis

- **Goal:** passage  $x$ 가 source model  $p_\theta$  (특정 LLM)에서 나왔는지 탐지.
- **Assumption:**  $\log p_\theta$ 를 평가할 수 있다(could be evaluated) — 즉 not a black-box setup (white-box: 소스 모델의 log probability 접근 필요).
- **Hypothesis:**  $p_\theta$ 의 샘플은  $\log p_\theta$  공간에서 negative curvature 영역(local maxima 근처)에 놓인다. 따라서 perturbed  $\tilde{x}$ 에 대해  $\log p_\theta(x) - \log p_\theta(\tilde{x})$ 가 상대적으로 크다(relatively large).
- **Why?**  $p_\theta$ 는 log-likelihood를 maximize하도록 학습되었고  $x$ 는  $p_\theta$ 에서 샘플되었기 때문.

### 방법: Curvature-based Criterion

Key idea는 training-free이면서 다른 LM(BERT, T5 등)을 활용해 random perturbation을 생성하는 curvature 기반 기준. 입력 문장이 주어지면 ① BERT/T5로 여러 perturbed version 생성 → ② 점수 계산 → ③ 점수 비교로 ChatGPT 산(産)인지 판정.

#### Perturbation Discrepancy Score

$$\mathbf{d}(x, p_\theta, q) = \log p_\theta(x) - \mathbb{E}_{\tilde{x} \sim q(\cdot|x)} [\log p_\theta(\tilde{x})]$$

원본 샘플의  $\log p_\theta(x)$ 와 perturbed  $\tilde{x}$ 의 기대 log probability 간의 **gap**.  $q(\cdot|x)$ 는 **perturbation function** —  $x$ 의 의미는 보존(**preserves the meaning**)하면서 표면 형태(syntax)만 약간 바꾼다. 예: **mask-filling LM(BERT, T5)**으로 일부 토큰을 마스킹하고 random sampling으로 채움(masked token prediction).

**Qualitative analysis:** model-generated text와 human-written text의 perturbation discrepancy 분포를 그려보면 둘 사이에 **large discrepancy**가 보인다 (모델 생성 쪽이 d가 큼).

★ 시험 핵심

d 점수의 판정 논리: **d가 크면(perturb하면 likelihood가 뚝 떨어지면) → local maximum 근처 → machine-generated**. d가 0 근처면(perturb해도 비슷하면) → human-written. + 3종 세트 암기: **zero-shot / training-free / not black-box(log  $p_\theta$  평가 가능 가정)**.

⚠ 함정 포인트 (T/F 대비)

- "DetectGPT requires training a classifier on labeled human/AI text pairs." → **False**. It is a **zero-shot, training-free** method based on a curvature criterion.
- "DetectGPT works in a fully black-box setting where only generated text is available." → **False**. It assumes  $\log p_\theta$  of the source model **can be evaluated**, i.e., NOT a black-box setup.
- "The perturbation function  $q(\cdot|x)$  is designed to change the meaning of  $x$ ." → **False**. It **preserves the meaning** while altering surface form (e.g., mask-filling with T5/BERT).
- "DetectGPT uses the source LLM  $p_\theta$  itself to generate perturbations." → **False**. Perturbations come from a **separate mask-filling LM** (BERT/T5);  $p_\theta$  is only used for scoring.

왜 perturbation 기반 점수 d가 "curvature"인가? 두 가지 도구를 결합해 보인다.

### Step 1: Directional Second Derivative — Taylor Series Expansion

Taylor expansion으로 방향 2차 미분  $\mathbf{z}^T H_f(x) \mathbf{z}$  추정

$$f(x + h\mathbf{z}) \approx f(x) + h\mathbf{z}^T \nabla f(x) + \frac{h^2}{2} \mathbf{z}^T H_f(x) \mathbf{z}$$

$$\Rightarrow \mathbf{z}^T H_f(x) \mathbf{z} \approx \frac{f(x + h\mathbf{z}) + f(x - h\mathbf{z}) - 2f(x)}{h^2}$$

$+h\mathbf{z}$ 와  $-h\mathbf{z}$  두 전개를 더하면 1차 항이 소거되어, **함수값의 합/차만으로** Hessian  $H_f$  방향 성분을 추정(finite difference).

### Step 2: Hutchinson's Unbiased Estimator for Trace

Hutchinson's trace estimator

$$\text{tr}(A) = \mathbb{E}_{\mathbf{z} \sim q_z} [\mathbf{z}^T A \mathbf{z}], \quad \mathbf{z} \sim q_z \text{ with } \mathbb{E}[z_i] = 0, \text{Var}(z_i) = 1$$

평균 0, 분산 1인 랜덤 벡터  $\mathbf{z}$ 에 대한 기대값으로 행렬의 trace를 비편향 추정.

### Step 3: 결합 (h = 1) + Symmetric Noise

두 식 결합 (h = 1)

$$-\text{tr}(H_f(x)) = \mathbb{E}_{\mathbf{z}} [2f(x) - f(x + \mathbf{z}) - f(x - \mathbf{z})]$$

노이즈 분포가 **symmetric**, 즉  $p(\mathbf{z}) = p(-\mathbf{z})$ 이면  $\mathbb{E}[f(x + \mathbf{z})] = \mathbb{E}[f(x - \mathbf{z})]$  이므로:

$$f(x) - \mathbb{E}_{\mathbf{z}} [f(x + \mathbf{z})] = -\frac{1}{2} \text{tr}(H_f(x))$$

좌변이 정확히 DetectGPT의 perturbation discrepancy 형태( $f = \log p_\theta, x + \mathbf{z} \leftrightarrow \tilde{x}$ ). **Hessian의 trace는 curvature를 포착하는 척도**이므로, 제안된 점수는 실제로 **(negative) curvature 정보를 이용하는 것**이다. local maximum에서는 Hessian이 negative definite  $\rightarrow \text{tr}(H_f) < 0 \rightarrow d = -\frac{1}{2} \text{tr}(H_f) > 0$ 으로 크게 나온다.

#### 교수 강조 (강의 발언)

"It is known that the trace of the Hessian is some measurement that captures the curvature of  $f$ , so we can conclude that the proposed detection score indeed utilizes the curvature information."

#### 함정 포인트 (T/F 대비)

- "The curvature interpretation requires computing the full Hessian matrix of  $\log p_\theta$ ."  $\rightarrow$  **False**. The trace is **estimated stochastically** via Hutchinson's estimator using only function evaluations (no explicit Hessian).
- "Hutchinson's estimator requires  $\mathbf{z}$  to be Gaussian."  $\rightarrow$  **False**. It only requires  $\mathbb{E}[z_i] = 0$  and  $\text{Var}(z_i) = 1$ ; the symmetry condition  $p(\mathbf{z}) = p(-\mathbf{z})$  is the extra assumption used to drop the  $f(x - \mathbf{z})$  term.
- "The perturbation discrepancy equals the (positive) trace of the Hessian."  $\rightarrow$  **False**. It corresponds to the **negative** trace (up to constant):  $f(x) - \mathbb{E}[f(x + \mathbf{z})] = -\frac{1}{2} \text{tr}(H_f)$ . Near a local maximum the trace is negative, so the score is positive/large.

## Setup

- **Dataset:** 6개 데이터셋 조합 — news articles, Wikipedia paragraphs, Reddit posts 등. **Machine-generated text**는 실제(**human-written**) 텍스트의 첫 **30 tokens**로 LLM을 **prompting**하여 **이어쓰게(continuation)** 구성.
- **Metric: AUROC** (Area Under Receiver Operating Characteristic curve) — binary classifier 평가의 표준 지표. confidence score로 샘플을 정렬하고, threshold를 움직이며 **false positive rate vs true positive rate** 곡선을 그려 그 아래 면적을 잰다.
- **Details:** perturbation 생성에 **T5-3B**(mask filling) 사용, 기대값 계산을 위해 **100개의 perturbed inputs** 생성.

## 결과

- **vs zero-shot baselines** (likelihood, rank, entropy 등): DetectGPT가 **state-of-the-art**.
- **vs supervised detectors** (RoB-base / RoB-Ig = AI/human 텍스트로 fine-tuning한 RoBERTa): **in-distribution**에서는 **on-par**, **새 도메인(out-of-distribution)**에서는 **DetectGPT가 훨씬 우수** — supervised는 학습 도메인 밖에서 일반화가 약하다.
- **Scalability:** 당시 최대 LLM인 **GPT-3, Jurassic-2**에도 적용 가능.

## Ablation: Perturbation의 중요성

- **Mask-filling 모델 크기:** 큰 perturbation 모델일수록 고품질 perturbation → 성능↑.
- **Perturbation 개수:** 충분한 개수가 중요하나, **100개 근처에서 효과는 거의 saturate**.

### ★ 시험 핵심

숫자 암기: 첫 **30 tokens** prompting / **T5-3B**로 perturbation / **100 perturbed inputs** / metric은 **AUROC**. 비교 구도: in-distribution → supervised와 on-par, **OOD(new domains)** → **DetectGPT 승**.

- "DetectGPT outperforms supervised RoBERTa detectors in every setting, including in-distribution." → **False**. In-distribution it is only **on-par**; the clear advantage is on **new domains (OOD)**.
- "Increasing the number of perturbations always improves performance proportionally." → **False**. The benefit **saturates around 100** perturbations.
- "AUROC is computed at a single fixed classification threshold." → **False**. It integrates the TPR-FPR curve over **all thresholds**.

# Fast-DetectGPT — Conditional Probability Curvature 와 340배 가속

Bao et al., "Fast-DetectGPT: Efficient Zero-Shot Detection of Machine-Generated Text via Conditional Probability Curvature", ICLR 2024.

## DetectGPT의 한계

입력  $x$  하나마다 **100개의 perturbed  $\tilde{x}$** 를 만들어야 함 = **T5 inference 100번** → perturb & evaluate의 **computational cost**가 매우 큼.

## Key Idea: Conditional Probability로 효율적 Sampling & Scoring

치환:  $p_\theta(\tilde{x}) \rightarrow p_\theta(\tilde{x} | x)$

$$p_\theta(\tilde{x} | x) = \prod_j p_\theta(\tilde{x}_j | x_{<j}), \quad p_\theta(x | x) = p_\theta(x)$$

핵심 차이: 보통의 autoregressive 분해는  $p(x_j | x_{<j})$ 처럼 **같은 시퀀스 자신**에 조건을 걸지만, 여기서는  $\tilde{x}_j$ 가 **원본  $x$ 의 prefix  $x_{<j}$** 에 조건 — 즉 토큰  $\tilde{x}_j$ 들은 **주어진  $x$ 에 대해 서로 독립적으로 예측(independently predicted given  $x$ )**된다.

### Conditional Probability Curvature

$$\mathbf{d}(x, p_\theta, q_\varphi) = \frac{\log p_\theta(x | x) - \tilde{\mu}}{\tilde{\sigma}}, \quad \tilde{\mu} = \mathbb{E}_{\tilde{x} \sim q_\varphi(\cdot | x)} [\log p_\theta(\tilde{x} | x)]$$

DetectGPT 점수와 같은 구조(원본 log prob – perturbed 기대 log prob)이지만 **분산 항  $\tilde{\sigma}$ 로 normalize**하는 것이 새로 추가됨.

## Conditional Independent Sampling — 왜 빠른가

- $\tilde{x}_j \sim q_\varphi(\tilde{x}_j | x_{<j})$ : **고정된  $x$** 가 주어지면, 다른 샘플된 토큰에 의존하지 않고 각 위치를 샘플.
- Conventional sampling**:  $N$ 개 샘플을 autoregressive하게 만들면  $N \times L$ 번의 **LLM inference** 필요.

- **Fast-DetectGPT**: 각 위치에서 원본 prefix 조건의 분포로부터  $N$ 개 토큰을 동시에 뽑으므로  **$L$ 번의 inference로 충분** (scoring도 동일). 한 번의 forward pass로 모든 위치의 분포를 얻고 거기서  $N$ 개씩 샘플.

#### 📌 교수 강조 (강의 발언)

"The key idea is that Fast-DetectGPT does not care about *what* is sampled from the distribution; it only focuses on calculating the log probability." — 샘플의 내용이 아니라 조건부 log prob 계산만 필요하므로 독립 샘플링이 가능.

## Connection to Likelihood & Entropy

**Sampling과 scoring에 같은 모델**을 쓰면, 이 점수는 baseline 방법들의 합과 연결된다: **Likelihood**( $\log p_\theta(x)$ ) + **Entropy**( $-\tilde{\mu}$ )의 합 형태.

## 실험

- Setup은 DetectGPT와 동일(dataset, models, metric).
- **속도**: DetectGPT **22 hours (A100)** vs Fast-DetectGPT **4 minutes  $\approx \times 340$  speedup**. 성능은 보존되거나 일부 케이스에서 **큰 폭으로 더 좋음**.
- **ChatGPT/GPT-4 탐지**: 소스 모델의 log prob에 접근할 수 없으므로 **surrogate(예: GPT-J)**로 sampling/scoring을 근사 — 표기 (modelA/modelB) = sampling용 A / scoring용 B. 이렇게 해도 ChatGPT·GPT-4가 쓴 문단을 성공적으로 탐지.

#### ★ 시험 핵심

Fast-DetectGPT 3대 포인트: ①  $p_\theta(\tilde{x})$ 를  $p_\theta(\tilde{x}|x)$ 로 치환(조건은 항상 **원본  $x$** ) ②  $N \times L \rightarrow L$  inferences ③ **22 hours  $\rightarrow$  4 minutes ( $\times 340$ )**, 성능은 유지~상회. + score에 **분산 normalization** 추가, 동일 모델 사용 시 **Likelihood + Entropy**와 연결.

#### ⚠ 함정 포인트 (T/F 대비)

- "Fast-DetectGPT speeds up DetectGPT by reducing the number of perturbation samples."  $\rightarrow$  **False**. The speedup comes from **conditional independent sampling/scoring** ( $L$  inferences instead of  $N \times L$ ), not from using fewer samples.

- "In Fast-DetectGPT, each sampled token  $\tilde{x}_j$  depends on previously sampled tokens  $\tilde{x}_{<j}$ ." → **False**. It is conditioned on the **original** prefix  $x_{<j}$ ; sampled tokens are **independent of each other** given  $x$ .
- "Fast-DetectGPT trades detection accuracy for speed." → **False**. It preserves DetectGPT's performance and even **outperforms it by a large margin** in some cases.
- "Detecting ChatGPT/GPT-4 text with Fast-DetectGPT is impossible because their log-probabilities are inaccessible." → **False**. A **surrogate model (e.g., GPT-J)** is used for sampling/scoring and detection still succeeds.

# ReMoDetect — Reward Model로 Aligned LLM 텍스트 탐지

Lee et al., "ReMoDetect: Reward Models Recognize Aligned LLM's Generations", NeurIPS 2024.

## Motivation

- 최근 LLM은 **RLHF**로 human preference에 맞게 fine-tuning됨 (InstructGPT 그림 인용). 여기서 **reward model**은 **human preference**의 **proxy**.
- 핵심 관찰: reward model 학습 데이터는 사람이 쓴 글이 아니라 **LLM이 생성한 응답들 중 사람이 고른 (voted) 것** — 즉 **reward model is trained with LLM-generated texts, not human-written texts**.
- **Hypothesis**: LGT(LLM-Generated Text) distribution  $\neq$  Human-written text distribution  $\rightarrow$  reward model은 **LGT 쪽으로 biased**되어 있을 것.

## 관찰적 검증

- **t-SNE** (reward model 마지막 hidden embedding의 2차원 매핑): human 문장과 machine 문장이 **명확히 분리** (특히 Reuters 데이터셋).
- **Reward score 분포**: machine-generated text가 human보다 **더 높은 reward score** — LLM 자체가 reward를 maximize하도록 fine-tuning되었기 때문.

## Key Idea + 두 가지 Training Components

이 **reward model**을 그대로 **LGT 탐지기로 사용**하고, 효과를 높이기 위해 두 가지를 추가 학습:

- 1 **Continual Preference Tuning** — LGT/human text 쌍으로 reward model을 다시 fine-tuning. **주의**: 직관과 반대로 **LGT를 preferred 쪽으로** 두어 human과 LGT의 reward score 격차를 더 벌린다. 추가 fine-tuning 중 기존 reward modeling 정보를 잃는 **forgetting issue**는 **replay buffer**(중요 데이터 저장 후 regularization)로 완화.
- 2 **Reward Modeling with Mixed Responses** — human-written text를 **LLM으로 부분적으로 rephrase**한 mixed text를 만들고, 이를 **median preference**로 사용: **Preference: LLM > Mixed > Human**. 중간 단계 문장을 넣어 탐지기가 **더 나은 decision boundary**를 학습.

## 실험

- **Trained reward model**: OpenAssistant의 **reward-model-deberta-v3-large-v2 (700M)**  
— DeBERTa 기반 오픈소스. 학습 데이터: **HC3** (Human/ChatGPT-3.5 pairs) **4,400 samples**.
- **평가**: MGTBench, Fast-DetectGPT Bench. **Unseen domains**(PubMed, XSum, WritingPrompt, Essay, Reuters)와 **unseen LLMs**(Llama, Gemini, GPT-4, Claude, Phi ...)에 대해 평가.
- **결과**: Fast-DetectGPT benchmark·MGTBench 모두에서 **모든 baseline 대비 우수**. 단, **ReMoDetect는 training-based, Fast-DetectGPT는 zero-shot**이라는 점은 구분해야 공정한 비교. **상용 detector(GPTZero 등)보다도 우수** → 상용화 잠재력. **DPO-trained LLM**(reward model 없이 정렬된 모델)의 생성물에도 성공적으로 적용됨.

### 교수 강조 (강의 발언)

"In contrast to human intuition, the language-model-generated text will be the *preferred* one" (continual preference tuning에서) — 그리고 비교 해석 시 "We should clarify that ReMoDetect is actually a training-based approach, while Fast-DetectGPT is a zero-shot-based approach."

### ★ 시험 핵심

논리 체인 암기: reward model은 **LLM 생성문으로 학습** → LGT에 biased → **LGT가 human보다 높은 reward** → reward score 자체가 탐지 신호. 기법 2개: **Continual Preference Tuning(LGT preferred + replay buffer)**, **Mixed Responses(LLM > Mixed > Human)**.

### ⚠ 함정 포인트 (T/F 대비)

- "Reward models in RLHF are trained on human-written texts." → **False**. They are trained on **LLM-generated responses** ranked/selected by human annotators.
- "In continual preference tuning, human text is labeled as the preferred response." → **False**. Counter-intuitively, the **LLM-generated text is set as preferred** to widen the reward gap.
- "ReMoDetect is a zero-shot detector like Fast-DetectGPT." → **False**. It is a **training-based** approach (fine-tunes a reward model).

- "ReMoDetect fails on DPO-trained LLMs because they have no reward model." → **False**. Experiments show it is **successfully applicable to DPO-trained models** as well.
- "Mixed responses are ranked above pure LLM generations." → **False**. The order is **LLM > Mixed > Human**; mixed texts serve as the **median** preference.

## Watermarking — Hard Red List, One Proportion z-Test, Soft Method

Kirchenbauer et al., "A Watermark for Large Language Models", ICML 2023. 실서비스 사례: SynthID by Google DeepMind.

### 정의

**Watermark in LLM: hidden pattern in text that is imperceptible to humans, while making the text algorithmically identifiable as synthetic.** — 사람 눈에는 품질 저하가 거의 없지만, 알고리즘으로는 LLM 산출물임을 판별 가능. 두 가지 key components: ① **how to insert** ② **how to detect**.

아이디어 개관(SynthID 발표 인용): LLM의 **output probability**를 **살짝 조정(adjusting)**해 생성 텍스트의 분포를 바꾸되, **원래 텍스트 품질은 최대한 보존**. 직관적으로 vocabulary를 **green token set / red token set**으로 나누고 **green set에서만 생성**하면, 워터마크된 응답은 대부분 green 토큰으로 구성되고 (탐지 가능), 일반 텍스트는 green/red가 반반쯤 섞인다.

### Algorithm 1: Generation with Hard Red List

- 1 표기:  $s^{(-N_p)}, \dots, s^{(-1)}$ 는 **prompt**,  $s^{(0)}, \dots, s^{(T)}$ 는 generated tokens. t번째 위치에서 next-token 확률분포를 얻음.
- 2 직전 토큰  $s^{(t-1)}$ 에 **hash function**을 적용해 그 해시로 random number generator를 seed.
- 3 그 난수로 vocabulary를 **green list / red list로 random partition**.
- 4 **green list에서만 다음 토큰을 샘플** (red list 토큰은 확률 0 — 절대 생성 안 됨).

### 탐지: Null Hypothesis와 One Proportion z-Test

#### Null Hypothesis

$H_0$ : **The text sequence is generated with no knowledge of the red list rule.** — 직관: red list 규칙을 모르는 natural writer는 무작위로 구성된 red list를 **위반할 수밖에 없다**. 사람이  $T$

토큰을 쓰면서 red list를 한 번도 위반하지 않을 확률은 단  $1/2^T$ . 따라서 위반이 없으면  $\rightarrow H_0$  기각  
 $\rightarrow$  AI-generated로 판정.

### One Proportion z-Test

$H_0$ 가 참이면 green list 토큰 수  $|s|_G$ 의 기대값은  $T/2$ , 분산은  $T/4$ . 따라서:

$$z = \frac{2(|s|_G - T/2)}{\sqrt{T}}$$

$z$ 가 threshold  $\tau$ 를 넘으면  $H_0$  기각  $\rightarrow$  watermark 탐지(= LLM 생성으로 판정). 예:  $\tau = 4$ 이면 **false positive 확률**  $\approx 3 \times 10^{-5}$ . 또한 **16 tokens 이상**의 watermarked sequence는 모두 탐지 가능 —  $|s|_G = T$ 일 때  $z = 4$ 가 되는 최소  $T$ 가 16이기 때문 ( $z = 2(T - T/2)/\sqrt{T} = \sqrt{T} = 4 \Rightarrow T = 16$ ).

## Hard Red List의 한계 $\rightarrow$ Soft Method

- **한계**: 예측의 **confidence/entropy(즉 context)**를 전혀 고려하지 않음. 예: "Barack" 다음은 거의 결정적으로 "Obama"인데, "Obama"가 우연히 red list에 들어가면 생성이 금지되어 **품질 저하** 발생.  
 $\rightarrow$  context/confidence에 따라 **adaptive**하게 적용할 수 있을까?

### Soft Watermark — logit에 $\delta$ 더하기

$l^{(t)}$ 를 LLM 마지막 층 출력(logit)이라 하면  $p_k^{(t)} = \frac{\exp(l_k^{(t)})}{\sum_i \exp(l_i^{(t)})}$  (softmax). Soft method는

**green list 토큰의 logit에만 상수  $\delta$ 를 더해** 그 확률을 키운다:

$$p_k^{(t)} = \frac{\exp(l_k^{(t)} + \delta \mathbf{1}[k \in G])}{\sum_i \exp(l_i^{(t)} + \delta \mathbf{1}[i \in G])}$$

**Why it works**:  $p^{(t)} \approx 1$ 인 highly likely word는  $\delta$ 를 더한 후에도 **여전히 가장 큰 확률로 남아** 생성됨 ("Obama"는 red여도 생성 가능). Hard red list는 **green에  $\delta = \infty$ 를 더한 극한**으로 볼 수 있다. 알고리즘의 나머지(hash  $\rightarrow$  seed  $\rightarrow$  partition)는 동일, 탐지도 **같은 one proportion z-test**(임의의 green 비율  $\gamma$ 에 일반화) + 저자들은 green 토큰 수의 기대값·분산에 대한 **theoretical backup** 제공.

## 실험: Watermark 강도 vs 품질 Trade-off

- Setup: **OPT-1.3B**로 생성,  $\gamma = 0.5$ (green/red 동수),  $\delta = 2.0$ .

- Qualitative: top row — **high entropy/z-score**로 쉽게 탐지되면서도 **품질 저하 없음**. Lower row — **failure case**: 텍스트의 entropy가 낮아 watermark가 너무 약해(too weak) 탐지 실패.
- Trade-off 곡선(왼쪽 sampling, 오른쪽 beam search): **z-score↑ = 탐지 쉬움**, **perplexity↓ = 품질 좋음** — 둘은 trade-off. 하지만  **$z \approx 2 \sim 4$ 면 탐지에 충분**하고, 이 구간에선 **유의미한 품질 저하가 없다** → Google DeepMind가 실서비스(SynthID)에 채택한 이유.

#### 📌 교수 강조 (강의 발언)

"The important thing is that we should preserve the original quality of the text as much as possible while inserting this kind of watermark." — 그리고 trade-off 결론: "z values near 2 or 4 are sufficient to detect the LLM-generated text, and in these ranges there is no significant quality degradation. That's the reason why it is utilized by Google DeepMind as well."

#### ★ 시험 핵심

절차 암기: **이전 토큰 hash → RNG seed → green/red random partition → green에서 샘플 (hard) 또는 green logit +  $\delta$ (soft)**. 탐지:  $z = 2(|s|_G - T/2)/\sqrt{T}$ ,  $\tau = 4$ 일 때  $FP \approx 3 \times 10^{-5}$ , **최소 16 토큰**. Hard의 한계 = low-entropy 상황("Barack→Obama") 품질 저하 → Soft가 해결.

#### ⚠ 함정 포인트 (T/F 대비)

- "LLM watermarks are visible patterns that humans can spot in the text." → **False**. They are **imperceptible to humans** but algorithmically identifiable.
- "The green/red partition is fixed once for the whole vocabulary across all positions." → **False**. It is **re-randomized at every position**, seeded by a hash of the previous token.
- "Watermark detection requires access to the LLM's parameters or log-probabilities." → **False**. Detection only needs the **hash/partition rule and a z-test on green token counts** — no model access (DetectGPT와의 결정적 차이).
- "In the soft method, red list tokens can never be generated." → **False**. That is the **hard** red list. Soft method only **boosts green logits by  $\delta$** ; a highly likely red token (e.g., "Obama") can still be generated.

- "A higher z-score threshold improves both detectability and text quality." → **False**. There is a **trade-off**: higher  $z$  → easier detection but worse (higher) perplexity. Luckily  $z \approx 2-4$  suffices with no significant degradation.
- "Watermarking can be applied to detect text that was already generated by any LLM." → **False**. The watermark must be inserted **at generation time** by the model owner; it cannot detect pre-existing/unwatermarked text.

## 탐지 패러다임 총비 (Zero-shot vs Trained vs Watermarking)

| 구분    | DetectGPT (ICML'23)                                                                                               | Fast-DetectGPT (ICLR'24)                          | ReMoDetect (NeurIPS'24)                                     | Watermarking (ICML'23)                                          |
|-------|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------------------|
| 패러다임  | Zero-shot (training-free)                                                                                         | Zero-shot (training-free)                         | <b>Training-based</b> (reward model fine-tuning)            | 생성 시 패턴 삽입 + 통계 검정                                              |
| 핵심 신호 | Probability curvature — perturbation discrepancy<br>$\log p_{\theta}(x) - \mathbb{E}[\log p_{\theta}(\tilde{x})]$ | Conditional probability curvature (분산 normalize)  | Reward score — aligned LLM 글이 높은 reward                     | Green token 비율의 z-statistic                                     |
| 필요 자원 | 소스 모델 $\log p_{\theta}$ 접근 (not black-box) + mask-filling LM(T5-3B), 100 perturbations                            | (surrogate 가능한) sampling/scoring 모델, L번 inference | 학습 데이터 (HC3 4,400 pairs) + 오픈소스 reward model (DeBERTa 700M) | <b>생성 시점</b> 에 모델 제어 필요; 탐지는 모델 접근 불필요                          |
| 비용    | 매우 큼 (T5 100회/입력, 22h on A100)                                                                                    | 4 min (×340 faster)                               | 학습 1회 후 inference 저렴                                        | logit에 $\delta$ 더하기만 — 거의 0                                     |
| 강점    | OOD(new domain)에서 supervised보다 강함; GPT-3급 확장                                                                      | 속도 + 성능 보존/상회; ChatGPT/GPT-4도 surrogate로 탐지       | 벤치마크 전반 SOTA, 상용 모델보다 우수, unseen LLM/domain-DPO 모델에도 작동     | 품질 저하 거의 없이 통계적 보장(FP $\approx 3 \times 10^{-5}$ ), 16+ 토큰이면 탐지 |

|                   |                     |                                |                                             |                                                          |
|-------------------|---------------------|--------------------------------|---------------------------------------------|----------------------------------------------------------|
| 약<br>점/<br>한<br>계 | 계산 비용, white-box 가정 | (zero-shot 공통) 소<br>스/대리 모델 필요 | training-based라<br>zero-shot과 직접 비교<br>시 유의 | 모델 소유자가 미리<br>심어야 함; hard 방<br>식은 low-entropy<br>서 품질 저하 |
|-------------------|---------------------|--------------------------------|---------------------------------------------|----------------------------------------------------------|

|                            | Hard Red List                               | Soft Watermark                                               |
|----------------------------|---------------------------------------------|--------------------------------------------------------------|
| 방식                         | red list 토큰 생성 <b>완전 금지</b> (green에서만 샘플)   | green list 토큰 <b>logit에 +<math>\delta</math></b> (확률만<br>상향) |
| 관계                         | Hard = Soft의 $\delta \rightarrow \infty$ 극한 |                                                              |
| <b>Low-entropy 상<br/>황</b> | "Barack → Obama"가 막혀 품질 저하                  | 고확률 토큰은 red여도 생성 → 품질<br>보존                                  |
| 탐지                         | one proportion z-test                       | 동일한 z-test (임의의 $\gamma$ 로 일반화)                              |

★ 시험 핵심

Pick-all 단골 구도: **"어떤 방법이 training-free인가?"** (DetectGPT·Fast-DetectGPT O / ReMoDetect·supervised RoBERTa X), **"어떤 방법이 생성 시점 개입이 필요한가?"** (Watermarking만 O), **"어떤 방법이 source model의 log prob 접근을 가정하는가?"** (DetectGPT O / Watermarking 탐지는 X).

## Q 예상문제 (직접 기출 없음 — 기말 신규 범위)

### True / False

#### Q1. (True/False) 예상문제

DetectGPT is based on the observation that LLM-generated texts tend to lie in regions of negative curvature of the model's log probability function, i.e., near local maxima of  $\log p_\theta$ .

#### ▼ 정답 & 해설 보기

**True.** This is exactly the core hypothesis: since  $p_\theta$  is trained to maximize log-likelihood and  $x$  is sampled from  $p_\theta$ , machine-generated text sits near local maxima (negative curvature), so perturbing it decreases  $\log p_\theta$  noticeably.

#### Q2. (True/False) 예상문제

DetectGPT is a zero-shot detection method, and therefore it can be applied even when the log-probability of the source model cannot be evaluated.

#### ▼ 정답 & 해설 보기

**False.** Although DetectGPT is zero-shot (training-free), it explicitly **assumes  $\log p_\theta$  can be evaluated**, i.e., it is NOT a black-box setup. Zero-shot (no training) and black-box (no model access) are different notions — a classic trap.

#### Q3. (True/False) 예상문제

The perturbation discrepancy score of DetectGPT can be interpreted as an estimate of the negative trace of the Hessian of  $\log p_\theta$ , using Taylor expansion and Hutchinson's trace estimator under a symmetric noise distribution.

▼ 정답 & 해설 보기

**True.** Combining the finite-difference form of the directional second derivative ( $h = 1$ ) with Hutchinson's estimator, and assuming  $p(\mathbf{z}) = p(-\mathbf{z})$ , gives  $f(x) - \mathbb{E}[f(x + \mathbf{z})] = -\frac{1}{2}\text{tr}(H_f(x))$  — the trace of the Hessian captures curvature.

Q4. (True/False)

예상문제

Compared to supervised detectors such as fine-tuned RoBERTa, DetectGPT performs much better in the in-distribution setting but worse on new domains.

▼ 정답 & 해설 보기

**False.** It is the opposite: DetectGPT is only **on-par** with supervised detectors in-distribution, but **much better on new domains (out-of-distribution)**, where supervised detectors fail to generalize.

Q5. (True/False)

예상문제

Fast-DetectGPT reduces the sampling cost from  $N \times L$  LLM inferences to  $L$  inferences by sampling each perturbed token  $\tilde{x}_j$  conditioned on the original prefix  $x_{<j}$ , independently of other sampled tokens.

▼ 정답 & 해설 보기

**True.** Conditional independent sampling conditions every  $\tilde{x}_j$  on the fixed original  $x$  (not on previously sampled tokens), so all  $N$  samples at all positions are obtained from  $L$  forward passes — yielding the 22 hours  $\rightarrow$  4 minutes ( $\times 340$ ) speedup while preserving or even improving performance.

Q6. (True/False)

예상문제

In ReMoDetect, the reward model assigns higher reward scores to LLM-generated texts than to human-written texts, because the reward model was trained on LLM-

generated responses selected by human annotators rather than on human-written texts.

▼ 정답 & 해설 보기

**True.** This is the key observation (verified via t-SNE separation and reward score distributions): the reward model is biased toward LLM-generated text, and aligned LLMs are additionally fine-tuned to maximize that reward — so the reward score itself becomes a detection signal.

**Q7. (True/False)** 예상문제

In the hard red list watermarking scheme, the green/red partition of the vocabulary is determined by seeding a random number generator with a hash of the previously generated token, and the next token is sampled only from the green list.

▼ 정답 & 해설 보기

**True.** Hash of the previous token → seed RNG → random green/red partition at each position → sample only from the green list. A natural writer with no knowledge of the rule produces  $T$  tokens without violating it with probability only  $1/2^T$ .

**Q8. (True/False)** 예상문제

The soft watermarking method adds a constant  $\delta$  to the logits of red list tokens to decrease their probability, which prevents tokens like "Obama" after "Barack" from ever being generated.

▼ 정답 & 해설 보기

**False.** Two errors:  $\delta$  is added to the logits of **green** list tokens (red logits are unchanged), and the whole point is that a highly likely word with  $p^{(t)} \approx 1$  **remains the largest** even after the addition — so "Obama" CAN still be generated even if it falls in the red list. Banning red tokens entirely is the *hard* red list.

## Pick all that are correct

any wrong answer → 0 points

### Q9. (Pick all that are correct) 예상문제

Which of the following are correct about DetectGPT?

- a. The perturbation function  $q(\cdot|x)$  should preserve the meaning of  $x$  while changing its surface form, e.g., via mask-filling with T5.
- b. For each input, 100 perturbed versions are generated with T5-3B to estimate the expectation in the discrepancy score.
- c. A larger perturbation (mask-filling) model yields higher-quality perturbations and better detection performance.
- d. The machine-generated evaluation text was produced by prompting LLMs with the first 30 tokens of real human-written text.

#### ▼ 정답 & 해설 보기

정답: (a), (b), (c), (d) — 모두 정답

- (a) Correct.  $q(\cdot|x)$  is a meaning-preserving perturbation; mask-filling LMs like BERT/T5 are used.
- (b) Correct. T5-3B, 100 perturbed inputs per sample — also why the method is so expensive.
- (c) Correct. The ablation shows larger perturbing models matter; the number of perturbations saturates near 100.
- (d) Correct. Machine text = LLM continuation of the first 30 tokens of real text from the six datasets.

### Q10. (Pick all that are correct) 예상문제

Which of the following are correct about Fast-DetectGPT?

- a. It replaces  $p_\theta(\tilde{x})$  in DetectGPT with the conditional probability  $p_\theta(\tilde{x}|x)$ , where each  $\tilde{x}_j$  is predicted given the original prefix  $x_{<j}$ .
- b. Its conditional probability curvature score is normalized by a variance term, which the original DetectGPT score does not have.

- c. It reduces wall-clock time from 22 hours to 4 minutes on an A100, but at the cost of noticeably lower AUROC than DetectGPT.
- d. When the same model is used for both sampling and scoring, the score is related to the sum of the Likelihood and Entropy baselines.

▼ 정답 & 해설 보기

정답: (a), (b), (d)

- (a) Correct. Tokens  $\tilde{x}_j$  are independently predicted given the original  $x$ ; also  $p_\theta(x|x) = p_\theta(x)$ .
- (b) Correct. The variance normalization term  $\tilde{\sigma}$  is newly incorporated.
- (c) Incorrect. The  $\times 340$  speedup comes **without** performance loss — Fast-DetectGPT preserves DetectGPT's performance and even outperforms it by a large margin in some cases.
- (d) Correct. With a shared sampling/scoring model the score relates to Likelihood ( $\log p_\theta(x)$ ) + Entropy ( $-\tilde{\mu}$ ).

Q11. (Pick all that are correct) 예상문제

Which of the following are correct about ReMoDetect?

- a. In continual preference tuning, the LLM-generated text is treated as the preferred response, contrary to typical human intuition, in order to widen the reward gap.
- b. A replay buffer is used to mitigate the forgetting issue during the additional fine-tuning of the reward model.
- c. Mixed responses, created by partially rephrasing human text with an LLM, are used as the median preference: LLM > Mixed > Human.
- d. ReMoDetect cannot detect generations from DPO-trained LLMs, since DPO does not use an explicit reward model.

▼ 정답 & 해설 보기

정답: (a), (b), (c)

- (a) Correct. LGTs become the preferred one to further increase the human/LLM reward discrepancy.
- (b) Correct. Important reward-modeling data is replayed to regularize the model against forgetting.
- (c) Correct. The intermediate (mixed) preference level helps the detector learn a better decision boundary.
- (d) Incorrect. Experiments show ReMoDetect is **successfully applicable to DPO-trained models** as well, even though DPO requires no separate reward model.

**Q12. (Pick all that are correct)** 예상문제

Which of the following are correct about watermark detection with the one proportion z-test (hard red list, half green/half red)?

- The null hypothesis  $H_0$  states that the text sequence was generated with no knowledge of the red list rule.
- Under  $H_0$ , the number of green list tokens  $|s|_G$  has expected value  $T/2$  and variance  $T/4$ .
- The watermark is detected by rejecting  $H_0$  when the z-statistic  $z = 2(|s|_G - T/2)/\sqrt{T}$  exceeds a threshold  $\tau$ .
- With  $\tau = 4$ , the false positive probability is about  $3 \times 10^{-5}$ , and any watermarked sequence of 16 or more tokens can be detected.

▼ 정답 & 해설 보기

**정답: (a), (b), (c), (d) — 모두 정답**

- (a) Correct. A human writer is expected to violate the randomly constructed red list rule.
- (b) Correct. Each token is green with probability  $1/2 \rightarrow \text{Binomial}(T, 1/2)$ : mean  $T/2$ , variance  $T/4$ .
- (c) Correct. Rejecting  $H_0$  means the text is considered AI-generated.
- (d) Correct. 16 is the minimum  $T$  producing  $z = 4$  when  $|s|_G = T$  (since  $z = \sqrt{T}$  in that case).

### Q13. (Pick all that are correct) 예상문제

Which of the following are correct about the soft watermarking method and its trade-offs?

- a. The hard red list can be viewed as the limiting case of the soft method where  $\delta \rightarrow \infty$ .
- b. The motivation for the soft method is that the hard red list ignores the confidence/entropy of the prediction, e.g., "Barack" followed almost deterministically by "Obama".
- c. In the experiments (OPT-1.3B,  $\gamma = 0.5$ ,  $\delta = 2.0$ ), the failure cases occur on high-entropy text where the watermark becomes too strong.
- d. There is a trade-off between detectability (z-score) and quality (perplexity), but z-scores around 2–4 are sufficient for detection without significant quality degradation.

#### ▼ 정답 & 해설 보기

정답: (a), (b), (d)

- (a) Correct. Adding  $\delta = \infty$  to green logits makes red tokens impossible — exactly the hard rule.
- (b) Correct. Low-entropy continuations get blocked by the hard rule, degrading quality; the soft method fixes this adaptively.
- (c) Incorrect. Failure cases are **low-entropy** texts where the watermark is **too weak** to be detected (almost no token changes occur) — the top-row success cases are the **high-entropy** ones.
- (d) Correct. This favorable trade-off region is why Google DeepMind adopted watermarking (SynthID).

### Q14. (Pick all that are correct) 예상문제

Comparing the detection paradigms covered in this lecture, which of the following are correct?

- a. DetectGPT and Fast-DetectGPT are zero-shot (training-free) methods, while ReMoDetect is a training-based method.
- b. Watermarking requires intervention at generation time by the model owner, whereas DetectGPT-style methods can be applied to text after it has been generated.
- c. ReMoDetect outperforms commercial detectors such as GPTZero in the reported comparison.
- d. Watermark detection requires evaluating the log-probability of the source model, just like DetectGPT.

▼ 정답 & 해설 보기

정답: (a), (b), (c)

- (a) Correct. This caveat was explicitly emphasized when comparing ReMoDetect against Fast-DetectGPT.
- (b) Correct. The watermark is a hidden pattern inserted while generating; post-hoc methods like DetectGPT score already-written passages.
- (c) Correct. ReMoDetect shows much better performance than commercial models, suggesting commercial potential.
- (d) Incorrect. Watermark detection only needs the green/red partition rule (hash) and a **z-test on green token counts** — no access to model log-probabilities is required, unlike DetectGPT's not-black-box assumption.

## 3분 요약

| 개념                    | 한 줄 정리                                                                                                                                                                                                   |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Motivation</b>     | LLM 텍스트는 사람 글과 구분 불가 수준 → 리뷰 자동생성 등 misuse → 정량적 탐지 필요 (GPTZero: not perfect, but not bad)                                                                                                               |
| <b>DetectGPT</b>      | LLM 텍스트는 $\log p_\theta$ 의 local maxima(negative curvature)에 위치 → perturbation discrepancy $\log p_\theta(x) - \mathbb{E}[\log p_\theta(\tilde{x})]$ 가 크면 machine. Zero-shot·training-free·not black-box |
| <b>Curvature 해석</b>   | Taylor expansion + Hutchinson's trace estimator + symmetric noise → 점수 = $-\frac{1}{2}\text{tr}(H_f)$ , Hessian trace = curvature                                                                        |
| <b>실험</b>             | T5-3B로 100 perturbations, AUROC. zero-shot SOTA, OOD에서 supervised(RoBERTa)보다 우수, in-dist는 on-par                                                                                                         |
| <b>Fast-DetectGPT</b> | $p_\theta(\tilde{x}) \rightarrow p_\theta(\tilde{x} x)$ , conditional independent sampling으로 $N \times L \rightarrow L$ inferences. 22h → 4min (×340), 성능 유지~상회                                          |
| <b>ReMoDetect</b>     | Reward model은 LGT로 학습되어 LGT에 biased(높은 reward) → 탐지기로 사용. Continual preference tuning(LGT preferred + replay buffer) + Mixed responses(LLM > Mixed > Human). Training-based, 상용 모델보다 우수, DPO 모델에도 적용     |
| <b>Watermarking</b>   | 이전 토큰 hash → green/red partition → green만 샘플(hard) 또는 green logit+ $\delta$ (soft). 사람에겐 imperceptible, 알고리즘으로 identifiable                                                                              |
| <b>z-test 탐지</b>      | $z = 2( s _G - T/2)/\sqrt{T}$ ; $\tau=4 \rightarrow \text{FP} \approx 3 \times 10^{-5}$ , 16+ 토큰 탐지. $z \approx 2 \sim 4$ 면 품질 저하 없이 탐지 충분 (SynthID 채택 이유)                                               |